In [1]:
```
import pandas as pd
import numpy as np
```

Вычисление статистических показателей

In [3]:
```
df = pd.DataFrame(np.random.randint(1, 101, 35).reshape(7, 5), ['a', 'b', 'c', 'd', 'e', 'f',
                  ['France', 'Spain', 'Italy', 'Germany', 'Great Britain'])
```

In [5]:
```
df
```

Out[5]:

	France	Spain	Italy	Germany	Great Britain
a	6	29	7	28	16
b	53	55	3	74	35
c	21	92	80	9	69
d	8	29	85	80	86
e	40	48	40	52	50
f	37	68	75	53	58
g	48	66	35	42	100

In [7]:
```
df['France'].max()
```

Out[7]:
```
53
```

In [9]:
```
df['France'].min()
```

Out[9]:
```
6
```

In [11]:
```
df['France'].sum()
```

Out[11]:
```
213
```

In [13]:
```
df['France'].mean()
```

Out[13]:
```
30.428571428571427
```

In [15]:
```
df['France'].count()
```

Out[15]:
```
7
```

In [17]:
```
df.max()
```

Out[17]:
```
France            53
Spain             92
Italy             85
Germany           80
Great Britain    100
dtype: int32
```

In [19]:
```
df.describe()
```

Out[19]:

	France	Spain	Italy	Germany	Great Britain
count	7.000000	7.000000	7.000000	7.000000	7.000000
mean	30.428571	55.285714	46.428571	48.285714	59.142857
std	18.893183	22.581071	34.262989	24.824143	28.916135
min	6.000000	29.000000	3.000000	9.000000	16.000000
25%	14.500000	38.500000	21.000000	35.000000	42.500000
50%	37.000000	55.000000	40.000000	52.000000	58.000000
75%	44.000000	67.000000	77.500000	63.500000	77.500000
max	53.000000	92.000000	85.000000	80.000000	100.000000

In [21]:
```
df.describe().T
```

Out[21]:

	count	mean	std	min	25%	50%	75%	max
France	7.0	30.428571	18.893183	6.0	14.5	37.0	44.0	53.0
Spain	7.0	55.285714	22.581071	29.0	38.5	55.0	67.0	92.0
Italy	7.0	46.428571	34.262989	3.0	21.0	40.0	77.5	85.0
Germany	7.0	48.285714	24.824143	9.0	35.0	52.0	63.5	80.0
Great Britain	7.0	59.142857	28.916135	16.0	42.5	58.0	77.5	100.0

Нахождение уникальных значений

In [23]:
```
df.loc['a','France'] = 99
```

In [25]:
```
df
```

Out[25]:

	France	Spain	Italy	Germany	Great Britain
a	99	29	7	28	16
b	53	55	3	74	35
c	21	92	80	9	69
d	8	29	85	80	86
e	40	48	40	52	50
f	37	68	75	53	58
g	48	66	35	42	100

In [27]:
```
df['France'].unique()
```

Out[27]:
```
array([99, 53, 21,  8, 40, 37, 48])
```

In [29]:
```
# Количество уникальных значений
df['France'].nunique()
```

Out[29]:
```
7
```

In [31]:
```
# Показать количества появлений
df['France'].value_counts()
```

Out[31]:
```
France
99    1
53    1
21    1
8     1
40    1
37    1
48    1
Name: count, dtype: int64
```

Отбор пропущенных значений

В библиотеке pandas для обозначения пропущенных значений используется значение NaN библиотеки NumPy (от англ. Not a Number — не число).

In [33]:
```
df.loc['a', 'France'] = np.NaN
```

In [35]:
```
df
```

Out[35]:

	France	Spain	Italy	Germany	Great Britain
a	NaN	29	7	28	16
b	53.0	55	3	74	35
c	21.0	92	80	9	69
d	8.0	29	85	80	86
e	40.0	48	40	52	50
f	37.0	68	75	53	58
g	48.0	66	35	42	100

In [37]:
```
# Выбрать пропущенные значения
df[df['France'].isnull()].head()
```

Out[37]:

	France	Spain	Italy	Germany	Great Britain
a	NaN	29	7	28	16

In [39]:
```
df.dropna()
```

Out[39]:

	France	Spain	Italy	Germany	Great Britain
b	53.0	55	3	74	35
c	21.0	92	80	9	69
d	8.0	29	85	80	86
e	40.0	48	40	52	50
f	37.0	68	75	53	58
g	48.0	66	35	42	100

In [41]:
```
df.dropna(axis=1)
```

Out[41]:

	Spain	Italy	Germany	Great Britain
a	29	7	28	16
b	55	3	74	35
c	92	80	9	69
d	29	85	80	86
e	48	40	52	50
f	68	75	53	58
g	66	35	42	100

In [43]:
```
df.fillna(value='Something')
```

Out[43]:

	France	Spain	Italy	Germany	Great Britain
a	Something	29	7	28	16
b	53.0	55	3	74	35
c	21.0	92	80	9	69
d	8.0	29	85	80	86
e	40.0	48	40	52	50
f	37.0	68	75	53	58
g	48.0	66	35	42	100

In [45]:
```
df['France'].fillna(value=df['France'].mean())
```

Out[45]:
```
a    34.5
b    53.0
c    21.0
d     8.0
e    40.0
f    37.0
g    48.0
Name: France, dtype: float64
```

In [47]:
```
df
```

Out[47]:

	France	Spain	Italy	Germany	Great Britain
a	NaN	29	7	28	16
b	53.0	55	3	74	35
c	21.0	92	80	9	69
d	8.0	29	85	80	86
e	40.0	48	40	52	50
f	37.0	68	75	53	58
g	48.0	66	35	42	100

Группирование строк по значениям

In [49]:
```
dict = {'Company':['COMP1', 'COMP1', 'COMP2', 'COMP2', 'COMP3', 'COMP3'], 'Month':['JAN', 'FE
        'Salary':[2000, 1500, 3000, 5000, 2500, 3500]}
```

In [51]:
```
df = pd.DataFrame(dict)
```

In [53]:
```
df
```

Out[53]:

	Company	Month	Salary
0	COMP1	JAN	2000
1	COMP1	FEB	1500
2	COMP2	MAR	3000
3	COMP2	APR	5000
4	COMP3	May	2500
5	COMP3	JUN	3500

In [55]:
```
df.groupby('Company').agg({'Salary': ['sum'], 'Month':['count']})
```

Out[55]:

	Salary	Month
	sum	count
Company		
COMP1	3500	2
COMP2	8000	2
COMP3	6000	2

In [57]:
```
df.groupby(by='Company').count()
```

Out[57]:

	Month	Salary
Company		
COMP1	2	2
COMP2	2	2
COMP3	2	2

In [59]:
```
groupby_comp = df.groupby('Company')
```

In [61]:
```
groupby_comp.count()
```

Out[61]:

	Month	Salary
Company		
COMP1	2	2
COMP2	2	2
COMP3	2	2

In [63]:
```
groupby_comp.sum()
```

Out[63]:

Company	Month	Salary
COMP1	JANFEB	3500
COMP2	MARAPR	8000
COMP3	MayJUN	6000

In [65]:
```
groupby_comp.sum().loc['COMP3']
```

Out[65]:
```
Month     MayJUN
Salary      6000
Name: COMP3, dtype: object
```

In [67]:
```
groupby_comp.count()
```

Out[67]:

Company	Month	Salary
COMP1	2	2
COMP2	2	2
COMP3	2	2

In [69]:
```
groupby_comp.max()
```

Out[69]:

Company	Month	Salary
COMP1	JAN	2000
COMP2	MAR	5000
COMP3	May	3500

In [71]:
```
groupby_comp.min()
```

Out[71]:

Company	Month	Salary
COMP1	FEB	1500
COMP2	APR	3000
COMP3	JUN	2500

In [73]:
```
df.groupby('Company').describe()
```

Out[73]:

	Salary							
	count	mean	std	min	25%	50%	75%	max
Company								
COMP1	2.0	1750.0	353.553391	1500.0	1625.0	1750.0	1875.0	2000.0
COMP2	2.0	4000.0	1414.213562	3000.0	3500.0	4000.0	4500.0	5000.0
COMP3	2.0	3000.0	707.106781	2500.0	2750.0	3000.0	3250.0	3500.0

In [75]:
```
comp3 = df.groupby('Company').describe().transpose()['COMP3']
```

In [77]:
```
comp3
```

Out[77]:
```
Salary  count       2.000000
        mean     3000.000000
        std       707.106781
        min      2500.000000
        25%      2750.000000
        50%      3000.000000
        75%      3250.000000
        max      3500.000000
Name: COMP3, dtype: float64
```

Обход столбца в цикле

In [79]:
```
for x in df['Company'].loc[:2]:
    print(x)
```

```
COMP1
COMP1
COMP2
```

In [81]:
```
df['Company'].loc[:2]
```

Out[81]:
```
0    COMP1
1    COMP1
2    COMP2
Name: Company, dtype: object
```

Применение функции ко всем элементам в столбце

Для применения встроенной или заданной пользователем функции к каждому элементу в столбце использовать метод apply.

In [83]:
```
def times3(x):
    return x * 3
```

In [85]:
```
df['Salary'].apply(times3)
```

Out[85]:
```
0     6000
1     4500
2     9000
3    15000
4     7500
5    10500
Name: Salary, dtype: int64
```

In [87]:
```
df['Salary'].apply(times3).sum()
```

Out[87]:
```
52500
```

In [89]:
```
df['Salary'].apply(lambda x: x * 3)
```

Out[89]:
```
0     6000
1     4500
2     9000
3    15000
4     7500
5    10500
Name: Salary, dtype: int64
```

In [91]:
```
# Сгруппировать строки, применить функцию к группам
df.groupby('Company').apply(lambda x : x.count())
```

```
C:\Users\rezed\AppData\Local\Temp\ipykernel_3408\2986465745.py:2: DeprecationWarning: DataFram
eGroupBy.apply operated on the grouping columns. This behavior is deprecated, and in a future
version of pandas the grouping columns will be excluded from the operation. Either pass `inclu
de_groups=False` to exclude the groupings or explicitly select the grouping columns after grou
pby to silence this warning.
  df.groupby('Company').apply(lambda x : x.count())
```

Out[91]:

Company	Company	Month	Salary
COMP1	2	2	2
COMP2	2	2	2
COMP3	2	2	2

Конкатенация фреймов данных

In [93]:
```
# Создать фрейм данных
data_a = {'id': ['1', '2', '3'], 'first': ['Alex', 'Amy', 'Allen'], 'last': ['Anderson', 'Ack
df_a = pd.DataFrame(data_a, columns = ['id', 'first', 'last'])
```

In [95]:
```
# Создать фрейм данных
data_b = {'id': ['4', '5', '6'], 'first': ['Billy', 'Brian', 'Bran'], 'last': ['Bonder', 'Bla
df_b = pd.DataFrame(data_b, columns = ['id', 'first', 'last'])
```

In [97]:
```
df_a
```

Out[97]:

	id	first	last
0	1	Alex	Anderson
1	2	Amy	Ackerman
2	3	Allen	Ali

In [99]:
```
df_b
```

Out[99]:

	id	first	last
0	4	Billy	Bonder
1	5	Brian	Black
2	6	Bran	Balwner

In [101…
```
# Конкатенировать фреймы данных по строкам
pd.concat([df_a, df_b], axis=0)
```

Out[101…

	id	first	last
0	1	Alex	Anderson
1	2	Amy	Ackerman
2	3	Allen	Ali
0	4	Billy	Bonder
1	5	Brian	Black
2	6	Bran	Balwner

In [103…

```
# Конкатенировать фреймы данных по столбцам
pd.concat([df_a, df_b], axis=1)
```

Out[103…

	id	first	last	id	first	last
0	1	Alex	Anderson	4	Billy	Bonder
1	2	Amy	Ackerman	5	Brian	Black
2	3	Allen	Ali	6	Bran	Balwner

Слияние фреймов данных

Для того чтобы применить внутреннее соединение, использовать метод merge с параметром on, задающим столбец, по которому происходит слияние

In [105…

```
# Создать фрейм данных
employee_data = {'employee_id': ['1', ' 2', '3', '4'], 'name': ['Amy Jones', 'Allen Keys', 'A
dataframe_employees = pd.DataFrame(employee_data, columns = ['employee_id', 'name'])
```

In [107…

```
dataframe_employees
```

Out[107…

	employee_id	name
0	1	Amy Jones
1	2	Allen Keys
2	3	Alice Bees
3	4	Tim Horton

In [109…

```
# Создать фрейм данных
sales_data = {'employee_id': ['3', '4', '5', '6'], 'total_sales':[23456, 2512, 2345, 1455]}
dataframe_sales = pd.DataFrame(sales_data, columns = ['employee_id','total_sales'])
```

In [111…

```
dataframe_sales
```

Out[111…

	employee_id	total_sales
0	3	23456
1	4	2512
2	5	2345
3	6	1455

In [113…
```
# Выполнить слияние фреймов данных
pd.merge(dataframe_employees, dataframe_sales, on='employee_id')
```

Out[113…

	employee_id	name	total_sales
0	3	Alice Bees	23456
1	4	Tim Horton	2512

Метод merge по умолчанию выполняет операцию внутреннего соединения. Если необходимо сделать внешнее соединение, можно указать это при помощи параметра how

In [115…
```
# Выполнить слияние фреймов данных
pd.merge(dataframe_employees, dataframe_sales, on='employee_id', how='outer')
```

Out[115…

	employee_id	name	total_sales
0	2	Allen Keys	NaN
1	1	Amy Jones	NaN
2	3	Alice Bees	23456.0
3	4	Tim Horton	2512.0
4	5	NaN	2345.0
5	6	NaN	1455.0

Этот же параметр можно использовать для указания левого и правого соединений

In [118…
```
# Выполнить слияние фреймов данных
pd.merge(dataframe_employees, dataframe_sales, on='employee_id', how='left')
```

Out[118…

	employee_id	name	total_sales
0	1	Amy Jones	NaN
1	2	Allen Keys	NaN
2	3	Alice Bees	23456.0
3	4	Tim Horton	2512.0

In [120…
```
# Выполнить слияние фреймов данных
pd.merge(dataframe_employees, dataframe_sales, left_on='employee_id', right_on='employee_id')
```

Out[120…

	employee_id	name	total_sales
0	3	Alice Bees	23456
1	4	Tim Horton	2512

Метод merge поддерживает четыре основных типа соединений:

♦ внутреннее — вернуть только те строки, которые совпадают в обоих фреймах (например, вернуть любую строку, в которой значение employee id появляется одновременно В dataframe_employees И В dataframe_sales);

♦ внешнее— вернуть все строки в обоих фреймах; если строка существует в одном фрейме, но отсутствует в другом, то пропущенные значения заполнить значениями NaN (например, вернуть

все строки И В employee_id, И В dataframe_sales);

♦ левое — вернуть все строки из левого фрейма, но только те строки из правого фрейма, которые совпали с левым фреймом; пропущенные значения заполнить значениями NaN (например, вернуть все строки ИЗ dataframe_employees, но только те строки из dataframe saies, которые имеют значение для employee id, появляющееся В dataframe_employees);

♦ правое — вернуть все строки из правого фрейма, но только те строки из левого фрейма, которые совпали с правым фреймом; пропущенные значения заполнить значениями NaN (например, вернуть все строки из dataframe saies, но только те строки из dataframe_empioyees, которые имеют значение для empioyee_id, появляющееся В dataframe_sales).

In [122…
```
df1 = pd.DataFrame({
                      'user_name': ['James Brown', 'Jack White',
                                    'Jane Green'],
                      'country': ['USA', 'USA',
                                  'France']},
                    index=['ind1', 'ind2', 'ind3'])

df2 = pd.DataFrame({'order_id': [114, 235, 432],
                      'user_id': [2, 4, 3],
                      'order_date': ['2020-02-11', '2020-02-11',
                                     '2020-02-15'],
                      'OS': ['Android', 'iOS', 'Android']},
                     index=['ind1', 'ind4', 'ind3'])
```

In [128…
```
df1
```

Out[128…

	user_name	country
ind1	James Brown	USA
ind2	Jack White	USA
ind3	Jane Green	France

In [130…
```
df2
```

Out[130…

	order_id	user_id	order_date	OS
ind1	114	2	2020-02-11	Android
ind4	235	4	2020-02-11	iOS
ind3	432	3	2020-02-15	Android

In [132…
```
df1.join(df2)
```

Out[132…

	user_name	country	order_id	user_id	order_date	OS
ind1	James Brown	USA	114.0	2.0	2020-02-11	Android
ind2	Jack White	USA	NaN	NaN	NaN	NaN
ind3	Jane Green	France	432.0	3.0	2020-02-15	Android

In [134…
```
df2.join(df1)
```

Out[134...

	order_id	user_id	order_date	OS	user_name	country
ind1	114	2	2020-02-11	Android	James Brown	USA
ind4	235	4	2020-02-11	iOS	NaN	NaN
ind3	432	3	2020-02-15	Android	Jane Green	France

In [ ]: